

Data mining practice 10

```
admission <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/ex2data1.txt",  
header = F, col.names = c('exam1', 'exam2', 'admitted'))
```

Task 2

```
admission <- read.csv("C:/Users/silkj/Desktop/한동대학교/5학기/데이터 마이닝 실습/Data-Mining-Practicum/myR/ex2data1.txt",  
header = F, col.names = c('exam1', 'exam2', 'admitted'))
```

다시, 데이터를 가져온다.

데이터는 exam1 과 exam2, 그리고 admitted(입학 여부)를 가진 데이터프레임이며, 우리는 이 exam1과 exam2변수를 통해 admitted를 예측하고자 한다

```
admission <- admission %>%  
  mutate(exam1_sq = exam1^2,  
         exam2_sq = exam2^2,  
         exam1_exam2 = exam1 * exam2)
```

다항 특징을 생성한다, mutate로 새로운 변수들을 만들며, 기존 변수인 exam1과 exam2를 제공하거나, 두 변수를 곱한 값을 새로운 변수로 만든다. 이 feature은 기존 특징 외 새로운 추가 특징으로 사용된다, 로지스틱 회귀는 선형적인 분류 경계를 만드는데, 다항 특징(비선형 특징)을 추가하면 모델이 더 복잡한 결정 경계를 학습할 수 있게 되어 데이터가 더 잘 분류될 가능성이 있다.

예를 들어,

`exam1^2` 이나 `exam2^2` 는 비선형성을 추가해 두 변수의 상호작용과 비선형 패턴을 학습하도록 돕는다.

```
# 추가 변수를 포함한 특징 데이터프레임 생성 및 정규화  
feature.df <- admission %>%
```

```
mutate(bias = 1) %>%
  select(bias, exam1, exam2, exam1_sq, exam2_sq, exam1_exam
2)
```

이제 추가한 변수들을 사용하여 feature df를 만든다.

이 feature df에 bias라는 상수 열을 추가하여 절편을 고려할 수 있도록 한다. 이후 select로 모델에 사용할 특징을 선택한다.

로지스틱 회귀 모델에서는 절편을 학습할 수 있어야 하므로 bias항을 추가한다.

```
feature.df[, 2:6] <- sapply(feature.df[, 2:6], function(x)
(x - mean(x)) / sd(x))
```

각 feature들을 정규화한다. 평균을 0, 분산을 1로 바꾸어 경사하강법이 더욱 효율적으로 작동되도록 한다. (이전 선형회귀에서와 같은 이유)

```
labelVector <- admission$admitted
```

종속 변수(레이블)를 저장한다. 모델의 예측 값과 실제 값을 비교하기 위해 이후 예측값과 labelvector과 비교하여 성능을 확인하게 된다

```
sigmoid <- function(x) {
  1 / (1 + exp(-x))
}
```

다음 시그모이드 함수를 정의한다. 로지스틱 회귀는 이진 분류 모델로, 예측값이 확률로 표현되어야 한다. 시그모이드 함수는 입력값을 0과 1 사이의 값으로 변환하여 특정 사건이 일어날 확률을 나타낸다. 이 확률을 기준으로 모델은 클래스(입학 여부)를 예측하게 된다.

(시그모이드 함수에 대한 추가적인 내용 첨부 혹은 Task1에서 명시)

```
h <- function(x, theta_vector) {
  sigmoid(theta_vector %*% x)
}
```

가설 함수를 정의한다. 위에서 정한 sigmoid함수를 이용해서 주어진 `theta_vector` 와 `x` 에 대해 시그모이드 함수를 적용하여 예측 확률을 계산한다. `theta_vector %*% x` 는 특징의 가중합을 의미하며, 이 값을 시그모이드 함수에 넣어 확률을 계산한다. 이를 통해 각 데이터 포인트가 입학할 확률을 구할 수 있다.

`theta_vector`는 각 특징에 대해 대한 가중치를 의미하며, 현재 사용된 `bias`, `exam1`, `exam2`, `exam1_sq`, `exam2_sq`, `exam1_exam2`에 대해 `vector = { $\theta_0, \theta_1, \theta_2, \theta_3, \theta_4, \theta_5$ }` 의 형태로 구성됨

이때 θ_0 은 bias로 절편에 대한 값이 들어가며, 그 외 θ 들은 각 feature의 가중치이다.

이러한 벡터값들은 모델이 학습하는 과정에서 최적의 값을 찾아가며 각 feature의 중요도를 의미한다.

이 `theta_vector`와 내적하는 `x`는 특정 데이터포인트의 실제 값으로서 로지스틱 회귀에서 계산하게 되는 가설함수인 $\theta_0 + \theta_1 * x_1 \dots$ 의 계산식을 따르게 된다.

```
costFunction <- function(theta_vector) {  
  hx <- apply(feature.df, 1, function(x) { h(x, theta_vector) })  
  costV <- labelVector * log(hx) + (1 - labelVector) * log(1 - hx)  
  -mean(costV)  
}
```

다음으로 예측 결과와 이전 `labelVector`에 저장한 실제 값의 차이를 측정하는 cost함수를 만든다

```
num_iter <- 1000  
m <- nrow(feature.df)  
alpha <- 1  
costDF <- data.frame(iter = 0,  
                      t0 = theta_vector[1],  
                      t1 = theta_vector[2],  
                      t2 = theta_vector[3],
```

```

t3 = theta_vector[4],
t4 = theta_vector[5],
t5 = theta_vector[6],
cost = costFunction(theta_vector))

for (i in 1:num_iter) {
  theta_update <- (t(as.matrix(feature.df)) %*% (apply(feature.df, 1, function(x) { h(x, theta_vector) }) - labelVector)) / m * alpha
  theta_vector <- theta_vector - theta_update[,1]
  costDF <- rbind(costDF, c(i, theta_vector, costFunction(theta_vector)))
}

```

다음으로 경사하강법을 통해 cost를 최소화하는 theta_vector을 찾는다. 이 과정을 통해 최적의 theta_vector을 찾아 구하고 데이터에 적합한 모델이 된다.

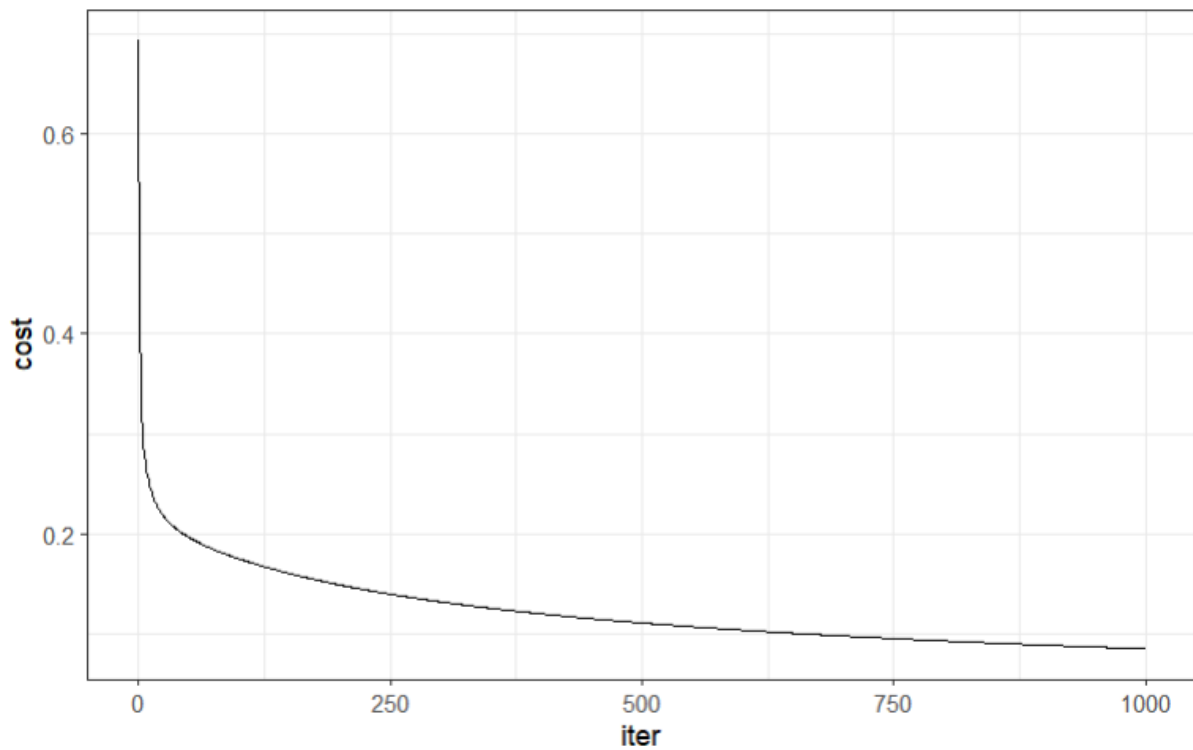
학습 곡선 시각화

```

ggplot(costDF, aes(x = iter, y = cost)) +
  geom_line() +
  ggtitle('Learning Curve with Additional Features') +
  theme_bw()

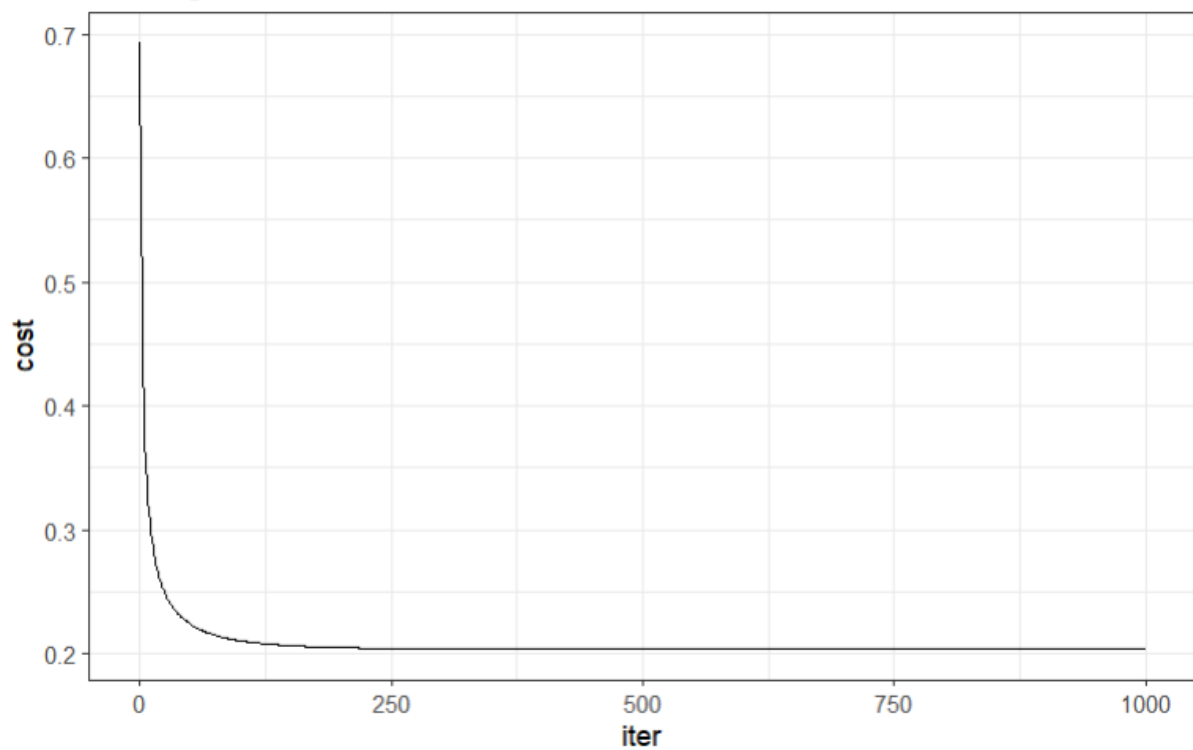
```

Learning Curve with Additional Features



학습 시행 횟수에 따른 cost값의 변화를 다음과 같이 표현하였다. 시행횟수가 250이 되기 전에 급격히 오차가 줄어들고 점점 수렴하다 시행이 끝나는 1000회에서 가장 낮다. 하지만 수렴하지는 않는 것 같다.

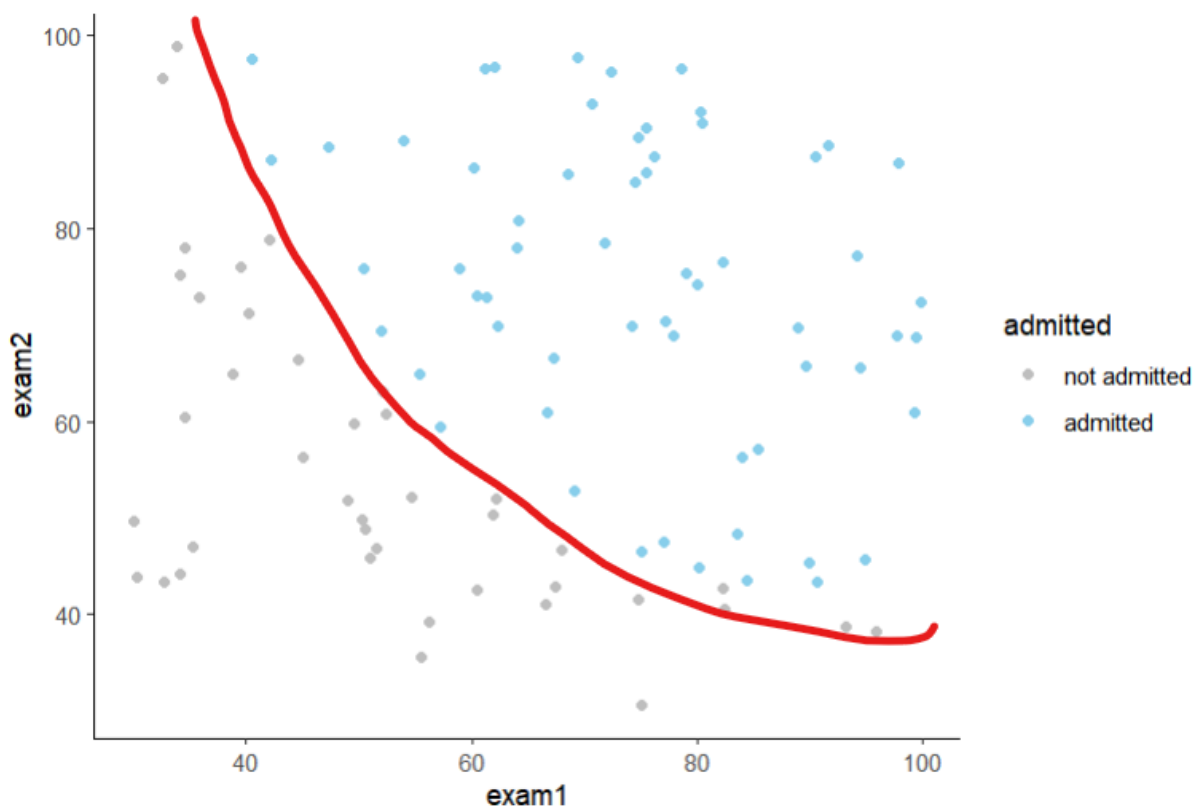
learning curve



입력변수를 추가하기 전과 비교하면 오차가 더욱 줄어들었고 그 대신 수렴하지는 않았다.

Task 3 문제2에서 학습한 모델의 decision boundary의 식을 표현하고 38페이지와 같은 형식의 그래프로 그려보아라.

decision boundary 결정경계 는 분류 모델에서 데이터를 분류하기 위한 경계선을 말한다.



exam1 과 exam2로 합격과 불합격을 나눈 그래프에서는 다음과 같은 붉은 선이 데이터를 분류하는 경계선이 될 수 있는데, 이를 구하기 위한 식과 실제 그래프상에서 표현이 가능하다.

로지스틱 회귀에서 결정 경계는 $\theta_0 + \theta_1 \times x_1 + \theta_2 \times x_2 = 0$

이 식으로 계산이 가능하다. 이 식에 우리가 구한 theta_vector을 대입하면

```
sprintf("%.2f + %.2f * x1 + %.2f * x2 = 0", theta_vector[1], theta_vector[2], theta_vector[3])
```

```
"4.68 + 3.09 * x1 + 2.61 * x2 = 0"
```

이러한 결정경계 식이 완성된다.

이제 이를 계산하고 시각화를 하면 다음과 같다

```
x1_vals <- seq(min(admission$exam1), max(admission$exam1),  
length.out = 100)  
x2_vals <- seq(min(admission$exam2), max(admission$exam2),  
length.out = 100)  
grid <- expand.grid(exam1 = x1_vals, exam2 = x2_vals)
```

우선 우리가 사용한 입력값이 다양하기 때문에, 실제 exam1과 exam2의 범위 안에서 100개의 값을 생성하고, 각각의 최소, 최대값을 가져와 $x_1 \times x_2$ 에 저장한다. 이후 x_1 과 x_2 의 모든 조합을 가지고 grid라는 데이터 프레임을 만든다

	exam1	exam2
1	30.05882	30.60326
2	30.76356	30.60326
3	31.46830	30.60326
4	32.17304	30.60326
5	32.87777	30.60326
6	33.58251	30.60326

다음과 같은 총 10,000개의 값이 포함되었음 이 데이터프레임은

exam1과 exam2가 가장 적을 때 부터 많을 때 까지의 결정 경계를 확인하기 위한 값들이 포함되어 있다.

```
grid <- grid %>%  
  mutate(exam1_sq = exam1^2,  
          exam2_sq = exam2^2,  
          exam1_exam2 = exam1 * exam2)
```

이제 grid에서 입력값인 x_1^2 , x_2^2 , x_1x_2 에 대해 새로운 변수를 만든다

```
grid_norm <- as.data.frame(sapply(grid, function(x) (x - mean(x)) / sd(x))))
grid_norm <- cbind(bias = 1, grid_norm) # 편향 추가
```

feature df와 동일하게, 정규화를 거친 후

```
prob_vals <- apply(grid_norm, 1, function(x) h(x, theta_vector))
```

각 grid 포인트에 대해 예측을 수행한다. 이 예측값은 앞서 만든 theta_vector과 동일하다 (최적화된 값)

```
grid$prob <- prob_vals
```

이제 예측 결과를 grid에 prob열에 저장한다.

	exam1	exam2	exam1_sq	exam2_sq	exam1_exam2	prob
1	30.05882	30.60326	903.5328	936.5597	919.8981	3.150922e-07
2	30.76356	30.60326	946.3966	936.5597	941.4653	3.740741e-07
3	31.46830	30.60326	990.2538	936.5597	963.0326	4.436464e-07
4	32.17304	30.60326	1035.1042	936.5597	984.5999	5.256243e-07
5	32.87777	30.60326	1080.9480	936.5597	1006.1672	6.221186e-07
6	33.58251	30.60326	1127.7851	936.5597	1027.7344	7.355805e-07
7	34.28725	30.60326	1175.6154	936.5597	1049.3017	8.688533e-07
8	34.99199	30.60326	1224.4391	936.5597	1070.8690	1.025232e-06
9	35.69672	30.60326	1274.2561	936.5597	1092.4363	1.208528e-06
10	36.40146	30.60326	1325.0664	936.5597	1114.0035	1.423150e-06
11	37.10620	30.60326	1376.8701	936.5597	1135.5708	1.674187e-06
12	37.81094	30.60326	1429.6670	936.5597	1157.1381	1.967507e-06
13	38.51568	30.60326	1483.4572	936.5597	1178.7053	2.309872e-06
14	39.22041	30.60326	1538.2408	936.5597	1200.2726	2.709061e-06
15	39.92515	30.60326	1594.0177	936.5597	1221.8399	3.174014e-06
16	40.62989	30.60326	1650.7878	936.5597	1243.4072	3.714995e-06

이제, 각 feature별 값과, 예측값인 prob값이 저장되어 있다.

```
# 시각화 - 결정 경계를 등고선으로 그리기
ggplot(admission, aes(x = exam1, y = exam2, color = as.factor(admitted))) +
  geom_point() +
  geom_contour(data = grid, aes(x = exam1, y = exam2, z = prob), breaks = 0.5, color = 'red') +
  scale_color_manual('admitted', values = c('grey', 'skyblue'), labels = c('not admitted', 'admitted')) +
  ggtitle(sprintf("%.2f + %.2f * x1 + %.2f * x2 = 0", theta_vector[1], theta_vector[2], theta_vector[3])) +
  theme_classic()
```

이제 exam1과 exam2에 대한 결정경계식을 적용한 시각화가 가능하다

